MANUAL DE IMPLEMENTAÇÃO

IoT Industrial em Tempo Real

Guia completo para sensores, MQTT, PostgreSQL/TimescaleDB, API e dashboard responsivo

Objetivo do manual: permitir que os alunos implementem e demonstrem a solução proposta utilizando prioritariamente software open source e recursos locais, sem necessidade de contratar serviços em nuvem ou licenças comerciais.

Disciplina: Tratamento e Armazenamento da Informação

Professor: Prof. Me. Nivaldo T. Marcusso

FACAMP - 2026



Versão acadêmica orientada a implantação local e custo zero.

Sumário orientativo

Seção	Conteúdo
1	Como usar este manual
2	O problema industrial e a arquitetura proposta
3	Banco de dados: tipos, aplicações e decisão arquitetural
4	Stack open source e estratégia de custo zero
5	Preparação do ambiente de desenvolvimento
6	Etapa 1 - Cadastro de ativos e sensores
7	Etapa 2 - PostgreSQL e TimescaleDB
8	Etapa 3 - DDL, hypertable, índices e políticas de dados
9	Etapa 4 - Broker MQTT com Eclipse Mosquitto
10	Etapa 5 - Simulador Python dos sensores
11	Etapa 6 - Subscriber MQTT e batch insert
12	Etapa 7 - API e dashboard responsivo
13	Etapa 8 - Anomalias, alarmes e evidências
14	Segurança, observabilidade, backup e recuperação
15	Teste de carga, dimensionamento e desempenho
16	Roteiro de apresentação ao professor
17	Troubleshooting
18	Apêndices: scripts completos e estrutura do projeto
19	Referências técnicas e licenças

Leitura recomendada: os capítulos 3 e 4 explicam por que este projeto combina banco relacional e banco temporal e como manter toda a solução sem custo de licenciamento. Os capítulos 6 a 13 reproduzem e ampliam as oito etapas do laboratório da animação.

1. Como usar este manual

Este material foi construído como complemento detalhado da animação interativa “IoT Industrial em Tempo Real”. A animação organiza o projeto em um fluxo de aprendizagem que parte do problema de negócio e chega à operação do banco, à ingestão MQTT e ao dashboard. O manual transforma esse fluxo em um roteiro executável, com decisões de arquitetura, comandos, scripts, critérios de validação e evidências que podem ser apresentadas ao professor.

O projeto deve ser tratado como uma prova de conceito acadêmica. Não é necessário adquirir sensores industriais reais: o aluno pode simular os dados em Python, transmitir as leituras via MQTT, persistir no banco e exibir os resultados em dashboards. Assim, o foco permanece no tratamento e armazenamento da informação, no desenho do banco e na demonstração do ciclo ponta a ponta.

- Percurso mínimo: banco + MQTT + simulador + ingestor + consultas + dashboard + alarme.
- Percurso completo: acrescentar API, segurança básica, observabilidade, backup/restore e teste de carga.
- Entrega técnica: código-fonte, scripts SQL, configuração do broker, prints/logs e README.
- Apresentação: demonstrar uma leitura normal, uma anomalia e a consulta histórica no banco.

Princípio de avaliação: o projeto deve provar funcionamento. Código sem execução, prints sem explicação ou um dashboard alimentado por dados estáticos não demonstram a solução completa.

2. O problema industrial e a arquitetura proposta

O cenário é um parque industrial com motores, compressores e utilidades que produzem sinais continuamente. O sistema precisa receber medições de temperatura, vibração, pressão, corrente, vazão e energia; armazená-las com timestamp; manter o histórico; identificar condições de atenção ou criticidade; e disponibilizar os dados em smartphones, tablets e notebooks.

A arquitetura didática é: Sensor/Simulador -> Gateway/Edge -> Broker MQTT -> Serviço de Ingestão -> PostgreSQL/TimescaleDB -> API/Dashboard. Essa separação é importante porque cada componente resolve um problema diferente: aquisição, transporte, persistência, consulta e apresentação.

Camada	Responsabilidade	Tecnologia sugerida
Sensor/Simulador	Produzir leituras com timestamp, unidade, qualidade e sequência	Python; opcionalmente ESP32/Wokwi

Gateway/Edge	Concentrar dispositivos, normalizar ou armazenar temporariamente	No laboratório, pode ser representado pelo próprio computador
Mensageria	Desacoplar produtores e consumidores	Eclipse Mosquitto / MQTT
Ingestão	Validar mensagens e gravar em lote	Python + paho-mqtt + psycopg
Banco	Cadastros + telemetria + alarmes	PostgreSQL + TimescaleDB; alternativa PostgreSQL puro
API	Expor dados de forma controlada	FastAPI + Uvicorn
Visualização	Cards, séries temporais, alarmes, filtros	Grafana OSS ou HTML/Chart.js

Em um ambiente real de automação industrial, protocolos como Modbus, OPC UA, Profinet ou Ethernet/IP podem existir entre sensores/CLPs e o gateway. Para a disciplina, esses protocolos podem ser abstraídos para reduzir a complexidade e concentrar o esforço no banco e no fluxo de dados.

3. Banco de dados: tipos, aplicações e decisão arquitetural

Uma das competências centrais do projeto é escolher o modelo de banco de dados a partir do padrão de acesso. A tecnologia não deve ser escolhida apenas pela popularidade: volume, velocidade de escrita, estrutura dos dados, consistência, consulta e retenção definem a melhor estratégia.

Tipo de banco	Modelo	Aplicações típicas	Adequação ao projeto IoT
Relacional (SQL)	Tabelas, chaves, integridade, transações	ERP, cadastro, faturamento, inventário	Excelente para ativos, sensores, usuários, thresholds e alarmes
Séries temporais	Timestamp como eixo principal; otimização por tempo/dispositivo	IoT, observabilidade, energia, mercado financeiro	Excelente para telemetria contínua, janelas e agregações
Documento	JSON/BSON flexível	Catálogos, perfis, conteúdo, microsserviços	Útil se o payload variar muito, mas não é necessário no MVP
Chave-valor	Chave -> valor	Cache, sessão, estado rápido	Pode complementar, mas não substitui o histórico temporal
Wide-column	Particionamento distribuído e alta escrita	Telemetria de escala extrema, eventos globais	Relevante em escala muito maior; aumenta complexidade
Grafo	Nós e arestas	Fraude, redes, dependências, recomendações	Útil para topologia industrial complexa, não necessário no

			MVP
Busca	Índice invertido	Logs, texto, pesquisa operacional	Complementar para busca de logs e eventos
Vetorial	Embeddings e similaridade	RAG, recomendação, IA generativa	Pode apoiar manutenção inteligente futura, fora do núcleo do projeto

3.1 Por que PostgreSQL + séries temporais?

O projeto contém dois perfis de dados. O primeiro é cadastral: um ativo possui nome, linha de produção e criticidade; um sensor pertence a um ativo e possui tipo e unidade. Esses dados exigem integridade referencial e são naturalmente relacionais. O segundo perfil é telemetria: milhares ou milhões de linhas formadas por sensor, instante e valor. Elas são ordenadas pelo tempo, escritas continuamente e consultadas por janelas temporais.

PostgreSQL atende muito bem a camada cadastral e também consegue armazenar séries temporais usando tabelas particionadas. O TimescaleDB é uma extensão do PostgreSQL que acrescenta abstrações específicas, como hypertables e funções temporais. Isso permite manter SQL e integridade do PostgreSQL, mas obter uma organização mais adequada a dados de tempo.

3.2 Tipos de dados recomendados

Campo	Tipo	Por quê
time	TIMESTAMPTZ	Preserva instante com fuso; ideal para sistemas distribuídos
sensor_id	VARCHAR/TEXT	Identificador legível e estável do ponto de medição
value	DOUBLE PRECISION	Medições numéricas com casas decimais
quality	SMALLINT	Qualidade de 0 a 100 com baixo consumo
seq	BIGINT	Sequência do dispositivo para rastreio/duplicidade
criticality	SMALLINT	Escala pequena, por exemplo 1 a 5
state	VARCHAR/TEXT com CHECK	Estados controlados: OPEN, ACK, CLOSED

metadata	JSONB opcional	Atributos variáveis sem criar muitas colunas

Evite armazenar valores numéricos como texto. Isso prejudica comparação, agregação e índice. Também evite timestamp sem timezone quando o projeto puder ser executado em máquinas com configurações diferentes. Para o laboratório, padronize UTC no simulador e TIMESTAMPTZ no banco.

3.3 Chaves, integridade e idempotência

O identificador do ativo deve ser único. Cada sensor deve referenciar um ativo existente por foreign key. Na telemetria, a combinação sensor_id + time pode funcionar como chave primária no laboratório; a sequência seq também pode ajudar a detectar mensagens duplicadas. Em um projeto maior, a estratégia precisa considerar sensores que enviam duas amostras no mesmo milissegundo ou reenvio fora de ordem.

Regra de modelagem: integridade referencial protege cadastros; idempotência protege a ingestão contínua. São problemas diferentes e ambos precisam ser tratados.

4. Stack open source e estratégia de custo zero

A implantação pode ser realizada inteiramente no notebook do aluno. Não é necessário contratar AWS, Azure, GCP, banco gerenciado, broker MQTT comercial ou plataforma paga de dashboards. A tabela abaixo separa ferramentas open source de componentes gratuitos com licenciamento próprio.

Componente	Escolha recomendada	Licença / condição	Custo para o laboratório
SGBD	PostgreSQL	PostgreSQL License - open source permissiva	R$ 0
Extensão temporal	TimescaleDB Apache 2 Edition	Apache 2.0 - open source	R$ 0, self-hosted
Opção temporal avançada	TimescaleDB Community	Recursos sob Apache 2 + Timescale License (TSL)	R$ 0 self-hosted, mas nem todos os recursos são OSI open source
Broker MQTT	Eclipse Mosquitto	EPL/EDL - open source	R$ 0
Cliente MQTT	Eclipse Paho	Open source	R$ 0
Linguagem	Python	PSF License	R$ 0
API	FastAPI	MIT	R$ 0
SGBD GUI	DBeaver Community	Free e open source	R$ 0
Dashboard	Grafana OSS	AGPLv3	R$ 0

Containers	Podman / Podman Desktop	Open source	R$ 0
Front-end alternativo	HTML/CSS/JavaScript + Chart.js	Chart.js MIT	R$ 0

Atenção à licença do TimescaleDB: para uma abordagem estritamente open source, utilize a edição Apache 2. A Community Edition é gratuita quando autogerenciada, porém inclui código sob Timescale License. Se houver dúvida ou dificuldade de instalação, o projeto também pode ser concluído com PostgreSQL puro e particionamento nativo.

4.1 Caminho recomendado para a turma

1. Instalar Podman Desktop ou PostgreSQL/Mosquitto diretamente no sistema operacional.

2. Executar PostgreSQL/TimescaleDB e Mosquitto localmente.

3. Usar DBeaver Community para administrar o banco.

4. Criar simulador e ingestor em Python.

5. Criar API em FastAPI.

6. Visualizar em Grafana OSS ou dashboard HTML/Chart.js.

7. Apresentar tudo na mesma máquina ou na rede local, sem serviços pagos.

Se a equipe preferir não usar containers, pode instalar cada componente diretamente. Containers apenas tornam a reprodução mais simples e evitam conflitos de versão.

5. Preparação do ambiente de desenvolvimento

5.1 Requisitos mínimos de hardware

- Processador 4 núcleos ou superior.
- 8 GB de RAM; 16 GB é recomendado para executar banco, Grafana e IDE simultaneamente.
- 10 GB de espaço livre para o laboratório.
- Windows 10/11, Linux ou macOS.
- Acesso local às portas 5432 (PostgreSQL), 1883 (MQTT), 8000 (API) e 3000 (Grafana), ajustáveis se necessário.

5.2 Estrutura de pastas

```
iot-industrial-facamp/
  README.md
  .env.example
  compose.yml
  sql/
    01_schema.sql
```

```
  02_seed.sql
  03_queries.sql
  04_backup_restore.md
 mosquitto/
  mosquitto.conf
 simulator/
  requirements.txt
  sensor_simulator.py
 ingestor/
  requirements.txt
  ingestor.py
 api/
  requirements.txt
  main.py
 dashboard/
  index.html        # se usar front-end próprio
 evidence/
  01_banco.png
  02_mqtt.png
  03_ingestao.png
  04_dashboard_desktop.png
  05_dashboard_mobile.png
  06_alarme.png
```

5.3 Variáveis de ambiente

```
# .env (não enviar senha real para repositório público)
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=iot_industrial
POSTGRES_USER=iot_app
POSTGRES_PASSWORD=troque_esta_senha
MQTT_HOST=localhost
MQTT_PORT=1883
MQTT_TOPIC=facamp/industrial/#
```

> **Segurança acadêmica:** mesmo em laboratório, não coloque senhas diretamente dentro do código Python. Use .env local e entregue apenas .env.example.

6. Etapa 1 - Cadastro de ativos e sensores

A primeira etapa cria a referência semântica para todas as leituras. Sem um cadastro consistente, um valor 72.4 não informa se representa temperatura, pressão ou vibração; também não indica a qual equipamento pertence. Por isso, a telemetria deve sempre chegar associada a um sensor previamente cadastrado.

Ativo	Linha/Setor	Criticidade	Sensores

MOTOR-07	Linha A	5 - alta	Temperatura e vibração
COMP-02	Utilidades	4 - alta	Pressão e temperatura
BOMBA-03	Linha B	3 - média	Corrente e vazão

6.1 DDL de cadastro

```
CREATE TABLE assets (
  id VARCHAR(60) PRIMARY KEY,
  name VARCHAR(120) NOT NULL,
  production_line VARCHAR(80) NOT NULL,
  criticality SMALLINT NOT NULL CHECK (criticality BETWEEN 1 AND 5)
);

CREATE TABLE sensors (
  id VARCHAR(80) PRIMARY KEY,
  asset_id VARCHAR(60) NOT NULL REFERENCES assets(id),
  sensor_type VARCHAR(40) NOT NULL,
  unit VARCHAR(20) NOT NULL,
  sample_seconds INTEGER NOT NULL CHECK (sample_seconds > 0),
  enabled BOOLEAN NOT NULL DEFAULT TRUE
);
```

6.2 Carga inicial

```
INSERT INTO assets(id,name,production_line,criticality) VALUES
('MOTOR-07','Motor principal 07','Linha A',5),
('COMP-02','Compressor 02','Utilidades',4),
('BOMBA-03','Bomba 03','Linha B',3);

INSERT INTO sensors(id,asset_id,sensor_type,unit,sample_seconds) VALUES
('S-M07-TEMP-01','MOTOR-07','temperature','C',2),
('S-M07-VIB-01','MOTOR-07','vibration','mm/s',2),
('S-C02-PRES-01','COMP-02','pressure','bar',5),
('S-C02-TEMP-01','COMP-02','temperature','C',5),
('S-B03-CURR-01','BOMBA-03','current','A',5),
('S-B03-FLOW-01','BOMBA-03','flow','L/min',5);
```

6.3 Validação

```
SELECT a.id AS asset, a.name, s.id AS sensor, s.sensor_type, s.unit, s.sample_seconds
FROM assets a
JOIN sensors s ON s.asset_id = a.id
```

```
ORDER BY a.id, s.id;
```

Evidência esperada: print do DBeaver mostrando os três ativos e os seis sensores; uma consulta JOIN demonstrando a relação; e um pequeno dicionário de dados no README.

7. Etapa 2 - PostgreSQL e TimescaleDB

7.1 Opção A - ambiente com containers

A forma mais reproduzível é executar os serviços em containers locais. Para evitar dependência de licença comercial do runtime de containers, o manual recomenda Podman/Podman Desktop. Em muitos casos, os mesmos arquivos Compose podem ser utilizados por Podman Compose.

```
# Verifique a instalação
podman --version
podman compose version

# Crie a rede e suba os serviços definidos no compose.yml
podman compose up -d

# Liste os containers
podman ps
```

7.2 Exemplo de compose.yml

```
services:
  db:
    image: timescale/timescaledb:latest-pg16
    environment:
      POSTGRES_DB: iot_industrial
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  mosquitto:
    image: eclipse-mosquitto:2
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro

  grafana:
    image: grafana/grafana-oss:latest
    ports:
      - "3000:3000"
    volumes:
      - grafana:/var/lib/grafana
```

```
volumes:
  pgdata:
  grafana:
```

Para uma política estritamente open source, valide a edição/licença do TimescaleDB instalada ou substitua o container pela edição Apache 2. Se a equipe optar por Community Edition, ela continua sem custo quando autogerenciada, mas a licença de parte dos recursos é TSL.

7.3 Teste do banco

```
psql -h localhost -p 5432 -U postgres -d iot_industrial

SELECT version();
CREATE EXTENSION IF NOT EXISTS timescaledb;
SHOW timescaledb.license;
```

O comando SHOW timescaledb.license ajuda a identificar o modo de licença. Se a prioridade didática for apenas custo zero, Community self-hosted pode ser utilizada respeitando sua licença. Se a prioridade for “somente open source OSI”, use a edição Apache 2 ou PostgreSQL puro.

8. Etapa 3 - DDL, hypertable, índices e políticas de dados

8.1 Tabela de telemetria

```
CREATE TABLE telemetry (
  time TIMESTAMPTZ NOT NULL,
  sensor_id VARCHAR(80) NOT NULL REFERENCES sensors(id),
  value DOUBLE PRECISION NOT NULL,
  quality SMALLINT NOT NULL DEFAULT 100 CHECK (quality BETWEEN 0 AND 100),
  seq BIGINT,
  PRIMARY KEY (sensor_id, time)
);
```

8.2 Transformação em hypertable

```
SELECT create_hypertable(
  'telemetry', 'time',
  chunk_time_interval => INTERVAL '1 day',
  if_not_exists => TRUE
);

CREATE INDEX IF NOT EXISTS telemetry_sensor_time_idx
ON telemetry(sensor_id, time DESC);
```

A hypertable funciona como uma tabela lógica. Internamente, os dados são organizados em chunks por intervalo de tempo. A vantagem didática é que o aluno continua escrevendo SQL normal, enquanto o mecanismo otimiza a organização das séries temporais.

8.3 Thresholds e alarmes

```
CREATE TABLE sensor_thresholds (
  sensor_id VARCHAR(80) PRIMARY KEY REFERENCES sensors(id),
  warn_low DOUBLE PRECISION,
  warn_high DOUBLE PRECISION,
  critical_low DOUBLE PRECISION,
  critical_high DOUBLE PRECISION
);

CREATE TABLE alarms (
  id BIGSERIAL PRIMARY KEY,
  sensor_id VARCHAR(80) NOT NULL REFERENCES sensors(id),
  opened_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  closed_at TIMESTAMPTZ,
  severity VARCHAR(10) NOT NULL CHECK (severity IN ('WARN','CRITICAL')),
  state VARCHAR(10) NOT NULL DEFAULT 'OPEN' CHECK (state IN ('OPEN','ACK','CLOSED')),
  value DOUBLE PRECISION NOT NULL,
  message TEXT NOT NULL
);
```

8.4 Consultas essenciais

```
-- Último valor de um sensor
SELECT time, value, quality
FROM telemetry
WHERE sensor_id='S-M07-TEMP-01'
ORDER BY time DESC
LIMIT 1;

-- Histórico das últimas 2 horas
SELECT time, value
FROM telemetry
WHERE sensor_id='S-M07-TEMP-01'
  AND time >= now() - INTERVAL '2 hours'
ORDER BY time;

-- Agregação por minuto
SELECT time_bucket('1 minute', time) AS minute,
       avg(value) AS avg_value,
       max(value) AS max_value,
       min(value) AS min_value
FROM telemetry
WHERE sensor_id='S-M07-TEMP-01'
  AND time >= now() - INTERVAL '2 hours'
GROUP BY minute
ORDER BY minute;
```

8.5 Índices: quando criar e quando evitar

Índices aceleram leitura, mas aumentam espaço e custo de escrita. Em telemetria, criar um índice para cada combinação imaginável pode reduzir o throughput. Comece pelo padrão real de consulta: sensor + tempo. Depois use EXPLAIN ANALYZE para justificar novos índices.

```
EXPLAIN ANALYZE
SELECT time, value
FROM telemetry
WHERE sensor_id='S-M07-TEMP-01'
  AND time >= now() - INTERVAL '60 minutes'
ORDER BY time DESC;
```

8.6 Retenção e camadas hot/warm/cold

No laboratório não é obrigatório implementar armazenamento frio, mas é importante explicar a estratégia. Dados “hot” são recentes e detalhados; dados “warm” podem ser agregados por minuto; dados “cold” podem ser agregados por hora ou dia. A política reduz espaço sem perder tendências de longo prazo. Se a edição escolhida não oferecer automação de retenção desejada, o aluno pode implementar jobs SQL ou cron e manter a solução em ferramentas abertas.

9. Etapa 4 - Broker MQTT com Eclipse Mosquitto

MQTT é um protocolo publish/subscribe leve. O sensor publica em um tópico sem conhecer o consumidor. O ingestor assina o padrão de tópicos e processa as mensagens. Isso desacopla o ritmo dos sensores do ritmo do banco.

9.1 Convenção de tópicos

```
facamp/industrial/planta01/linhaA/motor07/temperatura
facamp/industrial/planta01/linhaA/motor07/vibracao
facamp/industrial/planta01/utilidades/compressor02/pressao

# Assinatura ampla do ingestor
facamp/industrial/#
```

9.2 mosquitto.conf para laboratório local

```
listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
log_dest stdout
```

Importante: allow_anonymous true serve apenas para laboratório isolado. Na apresentação, explique

que produção exige autenticação, TLS e ACL por tópico.

9.3 Teste manual

```
# Terminal 1 - subscriber
mosquitto_sub -h localhost -p 1883 -t 'facamp/industrial/#' -v

# Terminal 2 - publisher
mosquitto_pub -h localhost -p 1883 \
  -t 'facamp/industrial/planta01/linhaA/motor07/temperatura' \
  -m '{"sensor_id":"S-M07-TEMP-01","asset_id":"MOTOR-07","ts":"2026-10-
21T13:40:05Z","value":72.4,"unit":"C","quality":100,"seq":1}'
```

10. Etapa 5 - Simulador Python dos sensores

10.1 Dependências

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# Linux/macOS: source .venv/bin/activate
pip install paho-mqtt python-dotenv
```

10.2 Estrutura da mensagem

Cada mensagem deve ser autodescritiva o suficiente para ser validada: sensor_id, asset_id, timestamp, value, unit, quality e seq. O timestamp deve ser produzido em UTC. A sequência permite acompanhar perda ou duplicidade de mensagens.

10.3 sensor_simulator.py

```
import json, random, time, os
from datetime import datetime, timezone
from paho.mqtt.client import Client

MQTT_HOST = os.getenv('MQTT_HOST','localhost')
MQTT_PORT = int(os.getenv('MQTT_PORT','1883'))

sensors = [
  {'sensor_id':'S-M07-TEMP-01','asset_id':'MOTOR-07','topic':'facamp/industrial/planta01/linhaA/motor07/
temperatura','base':68.0,'sigma':0.8,'unit':'C'},
  {'sensor_id':'S-M07-VIB-01','asset_id':'MOTOR-07','topic':'facamp/industrial/planta01/linhaA/motor07/
vibracao','base':2.4,'sigma':0.15,'unit':'mm/s'},
  {'sensor_id':'S-C02-PRES-01','asset_id':'COMP-02','topic':'facamp/industrial/planta01/utilidades/compressor02/
pressao','base':7.5,'sigma':0.08,'unit':'bar'}
]
```

```
client = Client()
client.connect(MQTT_HOST, MQTT_PORT, 60)
client.loop_start()
seq = 0
mode = 'normal'

try:
    while True:
        seq += 1
        for s in sensors:
            value = random.normalvariate(s['base'], s['sigma'])
            payload = {
                'sensor_id': s['sensor_id'],
                'asset_id': s['asset_id'],
                'ts': datetime.now(timezone.utc).isoformat(),
                'value': round(value, 3),
                'unit': s['unit'],
                'quality': 100,
                'seq': seq
            }
            client.publish(s['topic'], json.dumps(payload), qos=1)
            print(s['topic'], payload)
        time.sleep(2)
except KeyboardInterrupt:
    pass
finally:
    client.loop_stop()
    client.disconnect()
```

10.4 Como injetar anomalia

Uma forma simples é adicionar uma variável de modo e alterar o valor-base do sensor. Por exemplo, temperatura normal em 68 °C; atenção acima de 75 °C; crítico acima de 85 °C. O aluno pode criar uma tecla/comando, um parâmetro de linha ou um segundo script que publica valores anormais.

11. Etapa 6 - Subscriber MQTT e batch insert

O serviço de ingestão é o elo entre mensageria e banco. Ele deve validar os campos, rejeitar mensagens inválidas, acumular pequenas quantidades e persistir em transação. O batch reduz overhead de conexão e commit.

11.1 Dependências

```
pip install paho-mqtt psycopg[binary] python-dotenv
```

11.2 Regras de validação

- sensor_id deve existir em sensors.

- ts deve ser timestamp ISO-8601 válido.
- value deve ser numérico.
- quality deve ficar entre 0 e 100.
- unit deve ser compatível com o cadastro, quando essa validação for implementada.
- Mensagens duplicadas devem ser ignoradas ou atualizadas de forma determinística.

11.3 SQL de inserção idempotente

```
INSERT INTO telemetry(time,sensor_id,value,quality,seq)
VALUES (%s,%s,%s,%s,%s)
ON CONFLICT (sensor_id,time) DO NOTHING;
```

11.4 Estrutura simplificada do ingestor

```
import json, os, psycopg
from paho.mqtt.client import Client

BUFFER_MAX = 200
buffer = []
conn = psycopg.connect(
   host=os.getenv('POSTGRES_HOST','localhost'),
   dbname=os.getenv('POSTGRES_DB','iot_industrial'),
   user=os.getenv('POSTGRES_USER','iot_app'),
   password=os.getenv('POSTGRES_PASSWORD')
)

SQL = """INSERT INTO telemetry(time,sensor_id,value,quality,seq)
       VALUES (%s,%s,%s,%s,%s)
       ON CONFLICT (sensor_id,time) DO NOTHING"""

def flush():
   global buffer
   if not buffer: return
   with conn.cursor() as cur:
      cur.executemany(SQL, buffer)
   conn.commit()
   print('batch:', len(buffer))
   buffer = []

def on_message(client, userdata, msg):
   try:
      d = json.loads(msg.payload)
      buffer.append((d['ts'], d['sensor_id'], float(d['value']), int(d.get('quality',100)), d.get('seq')))
      if len(buffer) >= BUFFER_MAX:
         flush()
   except Exception as e:
      print('mensagem rejeitada:', e)

client = Client()
client.on_message = on_message
client.connect(os.getenv('MQTT_HOST','localhost'), int(os.getenv('MQTT_PORT','1883')), 60)
client.subscribe('facamp/industrial/#', qos=1)
```

```
client.loop_forever()
```

11.5 Geração de alarme

Existem duas estratégias aceitáveis: gerar o alarme no ingestor, imediatamente após validar a leitura, ou executar um serviço separado de regras. Para o laboratório, gerar no ingestor reduz componentes. A regra deve consultar sensor_thresholds e criar um registro em alarms quando o valor ultrapassar warn_high/critical_high.

12. Etapa 7 - API e dashboard responsivo

12.1 API com FastAPI

```
pip install fastapi uvicorn psycopg[binary] python-dotenv
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Endpoints mínimos sugeridos: /api/assets, /api/assets/{id}/current, /api/sensors/{id}/history?minutes=60 e /api/alarms?state=OPEN. As consultas devem ser parametrizadas; nunca monte SQL concatenando diretamente parâmetros recebidos na URL.

12.2 Exemplo de endpoint

```
from fastapi import FastAPI, Query
import psycopg, os

app = FastAPI(title='FACAMP IoT Industrial')

@app.get('/api/sensors/{sensor_id}/history')
def history(sensor_id: str, minutes: int = Query(60, ge=1, le=1440)):
    sql = """
      SELECT time, value, quality
      FROM telemetry
      WHERE sensor_id=%s
        AND time >= now() - (%s * interval '1 minute')
      ORDER BY time
    """
    with psycopg.connect(host=os.getenv('POSTGRES_HOST','localhost'),
                         dbname=os.getenv('POSTGRES_DB','iot_industrial'),
                         user=os.getenv('POSTGRES_USER','iot_app'),
                         password=os.getenv('POSTGRES_PASSWORD')) as conn:
        with conn.cursor() as cur:
            cur.execute(sql, (sensor_id, minutes))
            rows = cur.fetchall()
    return [{'time':r[0], 'value':r[1], 'quality':r[2]} for r in rows]
```

12.3 Dashboard - caminho A: Grafana OSS

1. Acesse http://localhost:3000 e altere a senha inicial.

2. Adicione PostgreSQL como Data Source usando o serviço local.

3. Crie um painel “Stat” para último valor da temperatura.

4. Crie um painel “Time series” para histórico de 60 minutos.

5. Crie uma tabela de alarmes abertos.

6. Configure refresh de 5 s para a demonstração.

7. Teste o layout em notebook, tablet e smartphone.

Grafana OSS é a opção mais rápida para a apresentação e mantém o foco na disciplina de banco de dados. Se o professor desejar evidenciar desenvolvimento web, mantenha também a API FastAPI e crie um front-end próprio com Chart.js.

12.4 Dashboard - caminho B: HTML + Chart.js

O front-end pode consumir a API com fetch() e reorganizar cards com CSS Grid. Defina media queries para telas menores. Em smartphone, use uma coluna; em tablet, duas colunas; em notebook, quatro. O mesmo backend deve atender os três formatos.

13. Etapa 8 - Anomalias, alarmes e evidências

13.1 Configuração de thresholds

```
INSERT INTO sensor_thresholds(sensor_id,warn_high,critical_high) VALUES
('S-M07-TEMP-01',75,85),
('S-M07-VIB-01',4.5,7.0),
('S-C02-PRES-01',8.2,9.0)
ON CONFLICT (sensor_id) DO UPDATE SET
 warn_high=EXCLUDED.warn_high,
 critical_high=EXCLUDED.critical_high;
```

13.2 Cenário de teste demonstrável

Momento	Temperatura	Estado esperado	Evidência
T0	68 °C	NORMAL	Dashboard verde/normal
T1	78 °C	WARN	Linha persistida + alarme WARN
T2	88 °C	CRITICAL	Alarme crítico + destaque no dashboard
T3	70 °C	NORMALIZADO	Fechamento do alarme conforme regra

13.3 Consulta de comprovação

```
SELECT time, sensor_id, value
FROM telemetry
WHERE sensor_id='S-M07-TEMP-01'
ORDER BY time DESC
LIMIT 20;

SELECT id, opened_at, severity, state, value, message
FROM alarms
ORDER BY opened_at DESC
LIMIT 10;
```

A apresentação deve mostrar a cadeia: simulador publica -> broker recebe -> ingestor registra -> banco contém -> alarme existe -> dashboard muda. Esse encadeamento é a prova mais forte de que a solução é realmente integrada.

14. Segurança, observabilidade, backup e recuperação

14.1 Usuário de aplicação com menor privilégio

```
CREATE USER iot_app WITH PASSWORD 'troque_esta_senha';
GRANT CONNECT ON DATABASE iot_industrial TO iot_app;
GRANT USAGE ON SCHEMA public TO iot_app;
GRANT SELECT, INSERT ON telemetry, alarms TO iot_app;
GRANT SELECT ON assets, sensors, sensor_thresholds TO iot_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO iot_app;
```

O usuário usado pela aplicação não precisa criar tabelas, criar usuários ou instalar extensões. A administração deve ficar em outra conta. Em ambiente de produção, MQTT também deve exigir credenciais e TLS.

14.2 Métricas mínimas

Métrica	O que revela	Como obter
Mensagens/s	Taxa de entrada	Contador no ingestor
Lag de ingestão	Diferença entre ts da mensagem e now()	SQL/API
Linhas por minuto	Volume persistido	COUNT + time_bucket
Erros/rejeições	Qualidade dos payloads	Logs
Tempo de consulta	Desempenho	EXPLAIN ANALYZE / logs

Disco	Crescimento do banco	SO/Podman/PostgreSQL
Alarmes abertos	Risco operacional	Tabela alarms

14.3 Backup e restore

```
# Backup lógico
pg_dump -h localhost -U postgres -d iot_industrial -Fc -f iot_industrial.dump

# Restauração em outro banco
createdb -h localhost -U postgres iot_industrial_restore
pg_restore -h localhost -U postgres -d iot_industrial_restore iot_industrial.dump
```

Critério de qualidade: backup que nunca foi restaurado é apenas uma hipótese. A equipe deve testar ao menos uma restauração em banco separado e registrar a evidência.

15. Teste de carga, dimensionamento e desempenho

15.1 Estimativa de volume

Para dimensionar, calcule pontos por segundo = número de sensores / intervalo de coleta. Com 2.000 sensores a cada 1 s, são 2.000 pontos/s e 172,8 milhões de pontos/dia. Em um laboratório, reduza a escala para dezenas de sensores ou aumente o intervalo. O objetivo é compreender a ordem de grandeza, não derrubar o notebook.

```
pontos_por_segundo = sensores / intervalo_segundos
pontos_por_dia = pontos_por_segundo * 86400
volume_bruto_gb_dia = pontos_por_dia * bytes_por_ponto / 1_000_000_000
```

15.2 Teste progressivo

1. Comece com 3 sensores a cada 2 s.

2. Suba para 20 sensores a cada 1 s.

3. Depois teste 100 sensores simulados.

4. Observe CPU, memória, batch/s e atraso de ingestão.

5. Use EXPLAIN ANALYZE nas consultas mais lentas.

6. Ajuste batch entre 100 e 1.000 mensagens e compare.

Não existe um batch ideal universal. Lotes maiores reduzem commits, mas aumentam latência e risco de perda em memória. Para demonstração, 100 a 500 é uma faixa simples e explicável.

16. Roteiro de apresentação ao professor

A apresentação pode ser organizada em 10 a 12 minutos para evitar que o grupo gaste tempo abrindo arquivos. Prepare os serviços antes e mantenha quatro janelas prontas: broker/log, simulador, DBeaver e dashboard.

Tempo	Demonstração	Mensagem-chave
1 min	Arquitetura	Separação entre aquisição, mensageria, banco e visualização
2 min	Modelo de dados	Por que cadastro é relacional e telemetria é temporal
2 min	MQTT + simulador	Mensagens reais geradas em tempo de execução
2 min	Banco	Hypertable, índices e consultas
2 min	Dashboard	Último valor + histórico + responsividade
2 min	Anomalia	WARN/CRITICAL persistido e exibido
1 min	Open source/custo	Stack local sem serviços pagos; explicação das licenças

16.1 Checklist de entrega

- README com arquitetura e como executar.
- compose.yml ou instruções de instalação local.
- Scripts SQL numerados.
- Código do simulador, ingestor e API.
- Configuração do Mosquitto.
- Evidências do banco e do MQTT.
- Print do dashboard em notebook, tablet e smartphone.
- Evidência de anomalia e alarme.
- Backup e restore testados.
- Tabela das ferramentas e licenças utilizadas.

17. Troubleshooting

Problema	Causa provável	Ação

Porta 5432 ocupada	Outro PostgreSQL local	Trocar porta externa para 5433 ou parar o serviço antigo
Mosquitto não recebe	Tópico/host/porta incorretos	Teste mosquitto_pub e mosquitto_sub antes do Python
FK sensor_id falha	Sensor não cadastrado	Execute seed antes do simulador
Erro de timezone	Timestamp sem UTC	Use datetime.now(timezone.utc).isoformat()
Duplicidade	Reenvio MQTT/QoS	Use chave idempotente e ON CONFLICT
Dashboard vazio	Consulta, datasource ou janela temporal	Execute SQL direto no DBeaver
API não abre no celular	localhost aponta para o próprio celular	Use IP local do notebook e libere firewall na rede privada
Container sem memória	VM do Podman limitada	Aumente memória da Podman machine ou rode menos serviços
Timescale não disponível	Extensão/edição incompatível	Use PostgreSQL puro com particionamento nativo

17.1 Alternativa 100% PostgreSQL sem TimescaleDB

Se o grupo não conseguir instalar TimescaleDB ou quiser eliminar qualquer ambiguidade de licença, pode implementar a telemetria em PostgreSQL puro usando particionamento por intervalo de tempo. A arquitetura geral permanece válida. O grupo deve explicar que a extensão foi substituída por recurso nativo do PostgreSQL.

```
CREATE TABLE telemetry_pg (
  time TIMESTAMPTZ NOT NULL,
  sensor_id VARCHAR(80) NOT NULL REFERENCES sensors(id),
  value DOUBLE PRECISION NOT NULL,
  quality SMALLINT NOT NULL,
  seq BIGINT
) PARTITION BY RANGE (time);

CREATE TABLE telemetry_2026_10
PARTITION OF telemetry_pg
FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

CREATE INDEX ON telemetry_2026_10(sensor_id,time DESC);
```

18. Apêndices: scripts completos e estrutura do projeto

18.1 SQL de verificação rápida

```
-- Quantidade de pontos por sensor
SELECT sensor_id, count(*) AS points, min(time) AS first_ts, max(time) AS last_ts
FROM telemetry
GROUP BY sensor_id
ORDER BY points DESC;

-- Lag médio de ingestão nos últimos 5 minutos
SELECT avg(extract(epoch FROM (now()-time))) AS avg_lag_seconds
FROM telemetry
WHERE time >= now() - interval '5 minutes';

-- Alarmes abertos
SELECT a.opened_at, a.severity, a.sensor_id, s.sensor_type, a.value, a.message
FROM alarms a
JOIN sensors s ON s.id=a.sensor_id
WHERE a.state='OPEN'
ORDER BY a.opened_at DESC;
```

18.2 README mínimo sugerido

```
# FACAMP - IoT Industrial em Tempo Real

## Objetivo
Monitorar ativos industriais com telemetria MQTT e banco temporal.

## Arquitetura
Sensor Simulator -> Mosquitto -> Ingestor -> PostgreSQL/TimescaleDB -> FastAPI/Grafana

## Como executar
1. Subir serviços
2. Executar SQL
3. Iniciar ingestor
4. Iniciar simulador
5. Abrir API/dashboard

## Cenário de demonstração
- Normal: 68 C
- Warn: > 75 C
- Critical: > 85 C

## Evidências
Inserir prints e descrição dos testes.

## Licenças
Listar PostgreSQL, TimescaleDB edition, Mosquitto, Python, FastAPI, Grafana OSS, Podman/DBeaver.
```

18.3 Critérios sugeridos para autoavaliação do grupo

Critério	Pergunta de controle
Requisitos	O grupo explica quais ativos, sensores, frequências e thresholds foram escolhidos?
Banco	A modelagem tem PK, FK, tipos corretos, índice e consulta temporal?
Ingestão	Os dados chegam via MQTT e são persistidos automaticamente?
Tempo real	O dashboard reflete mudanças sem recarga manual de arquivo estático?
Anomalia	É possível provocar WARN/CRITICAL e demonstrar a linha no banco?
Mobile	O dashboard funciona em três larguras?
Open source	As ferramentas e edições/licenças estão documentadas?
Operação	Há logs, backup/restore e explicação de segurança?

19. Referências técnicas e licenças

As referências abaixo são recomendadas para consulta durante a implementação. As licenças devem ser verificadas na versão instalada, principalmente em componentes que possuem edições diferentes.

- PostgreSQL - licença e documentação oficial: https://www.postgresql.org/about/licence/ e https://www.postgresql.org/docs/
- TimescaleDB - comparação entre edições Apache 2 e Community: https://docs.timescale.com/about/latest/timescaledb-editions/
- TimescaleDB - repositório e arquivos de licença: https://github.com/timescale/timescaledb
- Eclipse Mosquitto: https://mosquitto.org/
- Eclipse Paho MQTT Python: https://eclipse.dev/paho/
- Podman Desktop: https://podman-desktop.io/docs/
- DBeaver Community: https://dbeaver.io/
- FastAPI: https://fastapi.tiangolo.com/
- Grafana OSS: https://grafana.com/grafana/download/?edition=oss
- Chart.js: https://www.chartjs.org/

Conclusão: a solução pode ser executada localmente sem serviços pagos; o valor acadêmico está em justificar a arquitetura do banco e demonstrar o fluxo ponta a ponta.